

RNSiM: Efficient Deep Neural Network Accelerator Using Residue Number Systems

Arman Roohi^{†*}, MohammadReza Taheri^{*}, Shaahin Angizi[‡], Deliang Fan[§]

[†]Department of Computer Science and Engineering, University of Nebraska-Lincoln, Lincoln, NE, USA

^{*}Independent Researcher, Tehran, Iran

[‡]Department of Electrical and Computer Engineering, New Jersey Institute of Technology, Newark, NJ 07102, USA

[§]School of Electrical, Computer and Energy Engineering, Arizona State University, Tempe, AZ, USA

*aroohi@unl.edu

Abstract—In this paper, we propose an efficient convolutional neural network (CNN) accelerator design, entitled *RNSiM*, based on the Residue Number System (RNS) as an alternative for the conventional binary number representation. Instead of traditional arithmetic implementation that suffers from the inevitable lengthy carry propagation chain, the novelty of *RNSiM* lies in that all the data, including stored weights and communication/computation, are performed in the RNS domain. Due to the inherent parallelism of the RNS arithmetic, power and latency are significantly reduced. Moreover, an enhanced integrated intermodulo operation core is developed to decrease the overhead imposed by non-modular operations. Further improvement in systems' performance efficiency is achieved by developing efficient Processing-in-Memory (PIM) designs using various volatile CMOS and non-volatile Post-CMOS technologies to accelerate RNS-based multiplication-and-accumulations (MACs). The *RNSiM* accelerator's performance on different datasets, including MNIST, SVHN, and CIFAR-10, is evaluated. With almost the same accuracy to the baseline CNN, the *RNSiM* accelerator can significantly increase both energy-efficiency and speedup compared with the state-of-the-art FPGA, GPU, and PIM designs. *RNSiM* and other RNS-PIMs, based on our method, reduce the energy consumption by orders of $28.77\times$ and $331.897\times$ compared with the FPGA and the GPU platforms, respectively.

Index Terms—residue number system; processing-in-Memory; convolutional neural network; accelerator

I. INTRODUCTION

Machine learning has shown impressive performance and success in the past decades on various applications, such as image recognition tasks, achieving close to human-level perception rates. Notably, deep Convolutional Neural Networks (CNNs) offer significant advantages for large-scale datasets and complex NN architectures. However, the processing demands of deep CNNs spanning hundreds of layers face serious challenges for their tractability in memory and computational resources. This so-called CNN “power and memory wall” has motivated alternative approaches to improve CNN efficiency in software/hardware domains. In software-based approaches, the use of quantizing parameters and network binarization [1] have been explored extensively to eliminate the need for intensive multiplication-and-accumulation (MAC) operations. Recently, utilizing weights with low bit-width and activations reduce both model size, and computing complexity [2].

Recently using unconventional number systems advancements such as Residue Number System (RNS) [3] and adaptive Posit format [4] can lead to further performance improvements in a vast range of computationally intensive applica-

tions. RNS is a non-weighted number system whose unique features and number-theoretic properties can boost special-purpose computations' speed. Contrary to the conventional weighted number system, which suffers from the inevitable lengthy carry propagation chain, RNS limits inter-digit carry propagation by performing arithmetic operations, e.g., addition, on small residues independently and in parallel. This leads to shortening the carry propagation chain of addition and reducing partial product matrices' size in multiplication. Although these interesting features offer high-speed and low-power VLSI implementation in many applications like DSPs and cryptography, RNS is limited by the complexity of intermodulo arithmetic operations, i.e. magnitude comparison, sign detection, and scaling and the need to interface data with two's complement domain via converters. Therefore, energy-efficient and low latency hardware realization of these costly RNS-based components is crucial.

From the hardware point of view, to improve the computing efficiency of CNNs, extensive studies for developing deep learning accelerators using GPUs, FPGAs, and ASICs have been researched. However, within conventional isolated computing units and memory elements interconnected via buses, there are serious challenges, such as limited memory bandwidth channels, long memory access latency, and high leakage power consumption [5]. Therefore, reducing the computational complexity and the memory footprint for implementing an efficient CNN accelerator for energy- and area-constrained embedded devices is vital but challenging. Processing-in-Memory (PIM) architecture as a non-von Neumann architecture has been well designed and developed for different applications [5], [6], [7] as a possible way to address the aforementioned challenges. The main idea of PIM is to incorporate logic computation within memory units such that memory can process data internally that provides inherent parallel processing mechanisms exploiting large internal memory bandwidth. Considering the larger memory capacities of DRAM and off-chip data movement reduction as opposed to SRAM-based PIM, processing-in-DRAM platforms [8], [6], [9] have gained much more attention as a promising accelerator for different applications. DRISA [8] presents two alternative 3T1C- and 1T1C-based PIM techniques as CNN's accelerators and improves speed and energy-efficiency by $7.7\times$ and $15\times$ over GPUs, respectively. Nevertheless, the implementations of X(N)OR and addition-based tasks face multiple challenges,

making these platforms inefficient. This inefficiency comes from the intrinsic logic complexity of X(N)OR operation that is implemented by majority/AND/OR-based multi-cycle operations and required row initialization methods.

In this paper, we propose an energy-efficient RNS-based CNN accelerator entitled *RNSiM*. In summary, our major contributions in this paper can be listed as follows:

1) We introduce fully RNS-based computational layers to build CNNs, including convolution, activation, and pooling layers using a low-bit width feature of RNS to reduce the power and latency. 2) To alleviate the overhead caused by intermodulo operations, we develop a new efficient mixed-radix unit, which can be employed to perform sign detection, comparison, and scaling operations. 3) Further improvement in area and energy consumption is achieved by developing a new PIM accelerator for RNS-based convolutions based on DRAM computational sub-arrays. 4) Finally, system accuracy in different data precision and system performance is carried out in terms of speed and energy. Using RNSiM, we achieve a significant reduction in the power and delay at the cost of negligible accuracy degradation.

II. BACKGROUNDS

A. Residue Number System (RNS)

An RNS is characterized by a set of L pairwise prime moduli m_i for $i = \{1, 2, \dots, L\}$ with $L \geq 2$. The representable data in RNS correspond to the dynamic range M which is defined by the product of all the moduli. Any unsigned integer $X \in [0, M)$ can be uniquely represented using an L -tuple $(x_1, x_2, \dots, x_L)$, in which x_i is the least positive number of the division of X by m_i . The same residue representation also provides for the signed integer $\hat{X} \in [-M/2, M/2)$ considering the $\hat{X} = \langle X + \lfloor M/2 \rfloor \rangle_M - \lfloor M/2 \rfloor$ congruence. According to this analogy, the relationship between X and its unsigned counterpart for a given moduli-set can be expressed as:

$$\hat{X} = \begin{cases} X & \hat{X} \geq 0 \\ X - M & \hat{X} < 0 \end{cases} \quad (1)$$

Let A , B , and R be three integers with the residue representation $(a_1, a_2, \dots, a_L)$, $(b_1, b_2, \dots, b_L)$ and $(r_1, r_2, \dots, r_L)$, respectively, where R is obtained by performing the *arithmetic operation* $\otimes \in \{+, -, \times\}$ upon A and B . In the RNS domain, R can be computed by $(r_1, r_2, \dots, r_L) = (\langle a_1 \otimes b_1 \rangle_{m_1}, \langle a_2 \otimes b_2 \rangle_{m_2}, \dots, \langle a_L \otimes b_L \rangle_{m_L})$, where $\langle a_1 \otimes b_1 \rangle_{m_1}$ denotes the modulo operation $\otimes$ upon two residues a_i and b_i . The arithmetic operations in RNS are performed in parallel over the independent channels on relatively short operands compared to their two's-complement counterparts [3]. Unlike arithmetic operation $\otimes$, some operations, referred as *intermodulo operations*, like reverse conversion and sign identification, magnitude comparison, and scaling, cannot be performed in parallel RNS channels. The computation of these groups of operations can be conducted by employing a variety of algorithms that map a weighted integer number to its equivalent residue. One of the eminent algorithms is the Mixed Radix

Conversion (MRC), adopted in this paper. Based on MRC, a weighted integer number like X can be obtained from its residue equivalent as follows [10]:

$$X = \sum_{i=2}^b \left(v_i \left(\prod_{j=1}^{i-1} m_j \right) \right) + v_1 \quad (2)$$

where using the following recurrence relations, the mixed-radix coefficients $v_i = \langle F_{i,j} \rangle_{m_i}$ are obtained sequentially:

$$F_{i,j} = \begin{cases} x_j & i = 0 \\ (F_{i-1,j} - v_{i-1}) \langle m_{i-1}^{-1} \rangle_{m_j} & i > 0 \end{cases} \quad (3)$$

where $\langle m_a^{-1} \rangle_{m_b}$ is the multiplicative inverse of m_a in modulo m_b . For a 3-moduli set RNS, based on the above equations, the representation of weighted number X using MRC can be obtained as:

$$X = v_1 + m_1 v_2 + m_1 m_2 v_3 \quad (4)$$

The mixed-radix coefficients v_i of (4) can be computed according to the following equations.

$$\begin{aligned} v_1 &= x_1 \\ v_2 &= \langle (x_2 - x_1) \langle m_1^{-1} \rangle_{m_2} \rangle_{m_2} \\ v_3 &= \langle [(x_3 - x_1) \langle m_1^{-1} \rangle_{m_3} - v_2] \langle m_2^{-1} \rangle_{m_3} \rangle_{m_3} \end{aligned} \quad (5)$$

B. Related Works

Due to the abovementioned DNN's challenges and RNS's advantages, recently, researchers from both academic and industry exhibit the advantages of the RNS computing paradigm to accelerate compute-intensive applications [11], [12]. In [13], [14], the bit width of numbers was reduced using nested RNS resulting in efficient lookup table (LUT) implementations. However, this approach suffered from data movement between memory and computation units, besides it requires a relatively high memory amount. In [15], an RNS-based artificial neural network was implemented, in which neurons were modeled by ROMs. Although it improved the efficacy, the method's main disadvantage was its scalability, which was constrained by the number of input neurons and the required memory storage. In [16], the hardware complexity of MAC operations, specifically in the CONV layer, was reduced by approximate computation implemented using RNS representation. This work implemented only RNS-based MAC operation in the hardware domain. However, in order to function correctly, this partial RNS-based implementation required several conversions between Binary Number System (BNS) and RNS, which imposed performance overhead. In [17], CONV layers were implemented using RNS on FPGAs, and computations of the other NN layers, including activation and pooling, were performed in the software domain. Similar to [16], [18], computation in the RNS domain required N^1 Binary Number System (BNS) to (from) RNS conversions before (after) each CONV layer, which imposed significant resources and delay overhead. In [19], the authors proposed a memristor-based DNN using RNS, entitled RNSnet. Unlike the previous

¹The number of CONV layers determines N .

methods, RNSnet converted all the parameters into RNS and performs MAC operations, activation, and pooling functions in the RNS domain and avoiding BNS/RNS conversion in between. They used LUTs to implement partial multiplications, whereas LUT-based implementations could not be generalized for different channel bit-lengths, and they provide low pipeline ability. Besides, performing multiplications requires three add operations for each of the three channels, which increases delay considerably. The Res-DNN architecture proposed in [20] implemented all the main components of the convolutional layer in the RNS domain. It is inspired by the Eyeriss architecture and leverages Huffman-based coding to reduce memory usage. The main issue herein is that the direct module-by-module mapping, which required specialized hardware units to implement each of intermodule operations, i.e., sign detector and comparison to form activation, and pooling (MAX) layers, respectively. Leveraging dedicated individual components significantly reduces the overall performance improvement of the RNS-based network.

III. PROPOSED RNSiM ACCELERATOR

Since most of the CNN runtime is occupied by performing MACs, boosting MAC operations is vital for real-time applications. Herein, two main processes, including RNS-based multiplication/accumulation and customized PIM architecture, are exploited to accelerate MAC operations in convolutional layers. Nonetheless, unlike the MACs, which are carried out in parallel and independently over the RNS channels, the intermodule operations are required to perform activation, pooling, and normalization can not be performed in parallel channels. Since the computation of intermodule operations is conducted using moduli and residues of more than one channel, a large area and power-delay-product overhead are imposed to the RNS-based network. To mitigate this issue, we introduce an efficient approach to enhance hardware utilization of the proposed accelerator. Figure 1 depicts the general architecture for our RNSiM structure, which comprises Image/Kernel Banks, Pre-Processing, Reverse Converter, and RNS Processing Unit (RPU). The RPU is equipped with two components: CONV Computation Unit (CCU) and Inter-module Operation Core (IOC).

Here, we give an overview of the RNSiM accelerator's functionality. Initially, both pre-trained weights, in RNS form, and input feature maps (ifmap), in BNS form, are read and loaded in Kernel and Image sub-banks of memory, respectively ①. To take advantage of the promising features of RNS, RNSiM performs almost the entire computations in the RNS domain. Thus, in step ②, first, the pre-processing unit quantizes ifmaps and converts all the quantized values to RNS by using forward converter. Then ifmaps and weights are efficiently mapped into computational sub-arrays to perform multiplications and accumulations in the RNS domain on parallel channels ③. The RNS-based MAC operations are performed efficiently using the proposed DRAM-based PIM architecture within the CCU. The final output of the first CONV layer is then transferred to the IOC to perform activation function, normalization, and

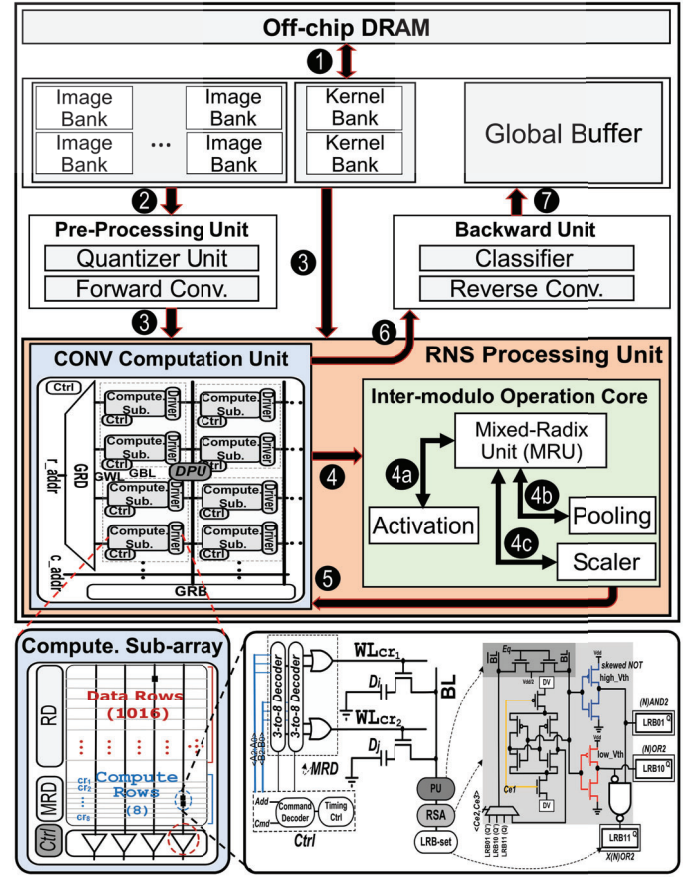


Fig. 1: General overview of the proposed RNSiM accelerator. The computational sub-array can be formed by various technologies, e.g. DRAM, with different and customized cell designs.

pooling ④. In step ④, to alleviate the potential overhead caused by the intermodule operations, Mixed-Radix Unit (MRU) produces mixed-radix coefficients for a target moduli set. These coefficients are leveraged to realize ReLU activation, via sign detection, ④a and MAX Pooling, by employing comparator, ④b and normalizing by using scaler ④c. The MRU uses the generated flag by Activation and Pooling blocks to enable/skip the normalization step. The scaling is adopted in this step to normalize the obtained value from ④b, to avoid the possible overflow in the next Conv layer. In step ⑤, CCU takes the IOC's output and performs the next CONV layer. After performing all the computations using CCU and IOC, to make the final decision, RPU's output is passed through Reverse Converter ⑥. Finally, the results are converted to the BNS by using a reverse converter unit and then classified by the activation function before storing in the embedded memories ⑦.

A. RNS Processing Unit (RPU) Architecture

We develop an RPU for the RNSiM as an independent component to accelerate the RNS-based CNN (RNS-CNN). The RPU, as shown in Fig. 1, consists of two sub-components, i.e., CCU and IOC, elaborated in the following subsections.

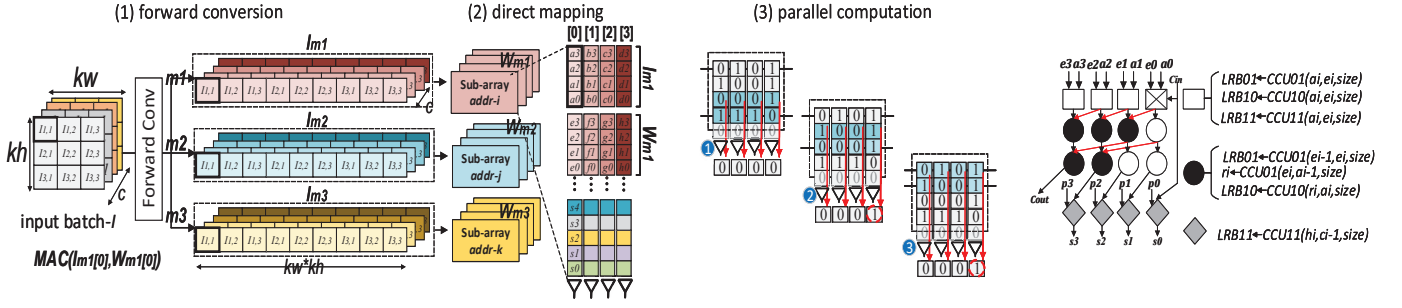


Fig. 2: The CCU data partitioning/ mapping method and parallel prefix adder computation. The RNSiM's instructions to process the summation in computational sub-arrays is given w.r.t. Table I.

B. CONV Computation Unit (CCU) Design

1) *DRAM Computational Sub-Array*: To implement the convolutional layers with summing blocks of parallel prefix adder (PPA) in the memory, we present a CCU on top of the typical DRAM hierarchy as depicted in Fig. 1. Each memory bank has multiple computational memory sub-arrays linked to two shared modules, i.e., a Global Row Decoder (GRD) and a Global Row Buffer (GRB). Each computational sub-array within CCU is designed to support full data-parallelization in-sub-array (N)AND, (N)OR, and XOR operations based on the physical address of input data. The row space (1024 rows) of the computational sub-array, as shown in Fig. 1, is divided into two distinct regions: 1-Data rows (1016), connected to a regular Row Decoder (RD) and 2-Computation rows ($8/Cr_1 - Cr_8$), connected to a Modified Row Decoder (MRD), to realize a 2-row activation for bulk bit-wise in-memory operations between operands. CCI's computational sub-array is motivated by Ambit [6], and ReDRAM [9], but enhanced and optimized to perform dual row activation mechanisms through charge-sharing among different DRAM cells. Each vertical bit-line (BL) is connected to a Precharge Unit (PU), a reconfigurable DRAM Sense Amplifier (RSA), and a Local Row Buffer Set. The RSA is developed with the regular cross-coupled inverters enhanced to two skewed inverters (high- V_{th} and low- V_{th}) and a 2-input NAND gate. The whole sub-array computation is managed through 3 main control signals (Ce_1, Ce_2, Ce_3). To perform the RNS-based MAC computation using the PPA scheme, two operands have to be copied to the compute rows. Here, the MRD activates the two copied rows (WL_{cr1} and WL_{cr2} in Fig. 1). The BL is initially precharged to $\frac{1}{2}V_{dd}$ by PU. Through setting the proper control signals, the input voltage of three inverters in RSA is given as $V_i = \frac{n \cdot V_{dd}}{2}$, where n is the number of DRAM cells storing logic '1'. The low- V_{th} inverter (red inverter in Fig. 1) connected to BL functions as a threshold detector by amplifying the deviation from $\frac{1}{4}V_{dd}$ and realizes a NOR2 function, whereas, the high- V_{th} inverter amplifies the deviation from $\frac{3}{4}V_{dd}$ and realizes a NAND2 function. A CMOS NAND gate is then considered to generate the XNOR2 function using NAND2 and OR2 logic results. A Local Row Buffer (LRB) set is designed and shared among multiple sub-arrays within each memory matrix to store the PIM operations result. In this way, LRB01, LRB02, and LRB03 are respectively used to save the (N)AND2, (N)OR2,

and XNOR2 outputs of RSA. It is noteworthy that the RNSiM's CCU is a general solution to process the parallel RNSed channels and could be implemented with other volatile and non-volatile memory candidates as shown in Section IV.

2) *Instruction Set*: While RNSiM is developed to be an independent high-performance and energy-efficient accelerator, programmers and system-level libraries need to access and utilize it. From a programmer's perspective, RNSiM is more of a third-party accelerator connected directly to the memory bus or through PCI-Express lanes rather than a memory unit; thus, it is integrated similar to that of Graphic-Processing Units (GPUs). In this way, we need to develop the backbone virtual machine and instruction set for parallel thread execution such that the programs could be translated at install time to the CCU's hardware instruction set tabulated in Table I. RNSiM receives one of the CCU00-CCU11 instructions from the memory controller and accordingly set Ce_1, Ce_2, Ce_3 control signals to implement one of the in-memory functions

3) *Hardware Mapping*: Figure 2 gives a clear picture of how the CCU data partitioning and mapping occur. As shown in step (1), the input batch with a size of $kw \times kh \times c$ is fed to the Forward Converter to provide three parallel channels for each integer number element corresponding to the target moduli set $\{m_1 = 2^{n+1} - 1, m_2 = 2^n - 1, m_3 = 2^n\}$, represented by Im_1, Im_2 , and Im_3 . This moduli set not only provides high performance arithmetic channels, but also enjoys effective intermodulo operation due to its remarkable number theoretic properties [21]. Each channel is then assigned to a certain sub-array address with the memory, where the corresponding kernel batch moduli is pre-stored W_{m1}, W_{m2} , and W_{m3} in step (2). For example, sub-array i to $i+3$ are allocated to compute the convolution between Im_1 and W_{m1} through summation implementation. To clarify these mapping, the structure of the sample 4-bit parallel prefix adder is demonstrated in Fig. 2, wherein each building block is effectively translated into RNSiM's ISA. All the preliminary calculation square blocks could be generated in a single memory cycle through CCU01, CCU10, and CCU11 and stored in LRB00, CCU10, and

TABLE I: The basic instructions of CCU.

opcode	operation	function	cycle(s)
CCU00 ($a, b, size$)	$b \leftarrow a$	Copy row a to Row b	1
CCU01 ($a, b, size$)	$a \cdot b$	AND2/NAND2	1
CCU10 ($a, b, size$)	$a + b$	OR2/NOR2	1
CCU11 ($a, b, size$)	$a \oplus b$	XOR2/XNOR2	1

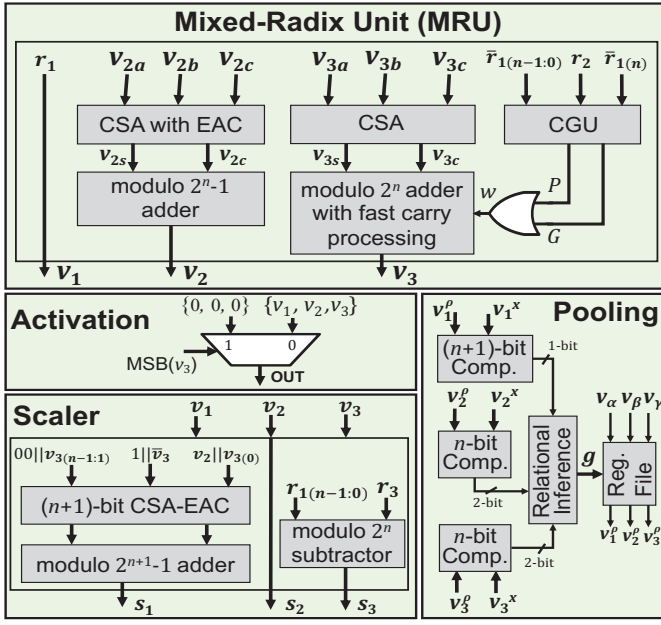


Fig. 3: The implementation details of Intermodulo Operation Unit (IOC).

CCU11, respectively. The carry operator (black circle), the main computation load of the adder, is implemented in 3 consecutive cycles with parallel in-memory OR2 and AND2 gates. Eventually, the CCU executes the final sum calculation (grey rhombus) in a single cycle issuing CCU11 instruction.

C. Intermodulo Operation Core (IOC) Design

As outlined earlier, in the RNS-CNN accelerators, activation, pooling, and normalization layers can present serious obstacles caused by the difficulty of intermodulo operations in RNS. The contribution in hand to reduce these operations' complexity proposes an Intermodulo Operation Core (IOC). As can be observed from Fig.3, the general schema of IOC comprises a Mixed-Radix Unit (MRU), an activation unit, a pooling unit, and a scaler unit. The IOC provides a massive hardware sharing between the intermodulo-based operations via MRU.

1) *Mixed-Radix Unit (MRU)*: To reduce the operations' complexity, we propose the MRU that produces the mixed-radix coefficients for the target moduli set. To realize activation, normalization, and pooling blocks, these mixed-radix coefficients are employed. Outsourcing these operations to the individual block MRU provides significant resource-saving for RNS-based DNN accelerators. A weighted number R with residue representation (r_1, r_2, r_3) can be described using MRC as $R = v_1 + (2^{n+1} - 1)v_2 + (2^{n+1} - 1)(2^n - 1)v_3$, where according to 5 and considering the fact that $\langle (2^{n+1} - 1)^{-1} \rangle_{2^{n-1}} = 1$, $\langle (2^{n+1} - 1)^{-1} \rangle_{2^n} = -1$, and $\langle (2^n - 1)^{-1} \rangle_{2^n} = -1$ the following hold.

$$\begin{aligned} v_1 &= r_1 \\ v_2 &= \langle (r_2 - r_1) \langle (2^{n+1} - 1)^{-1} \rangle_{2^{n-1}} \rangle_{2^{n-1}} = \langle r_2 - r_1 \rangle_{2^{n-1}} \\ v_3 &= \langle [(r_3 - r_1) \langle (2^{n+1} - 1)^{-1} \rangle_{2^n} - v_2] \langle (2^n - 1)^{-1} \rangle_{2^n} \rangle_{2^n} \\ &= \langle r_3 - r_1 + v_2 \rangle_{2^n} \end{aligned} \quad (6)$$

where v_2 and v_3 can be obtained and simplified in a quite similar way to equations (19) and (22) in [22]. Although the generated signals are only used to identify the sign, we highly exploit them to form the scaler and the comparator.

$$\begin{aligned} v_2 &= \langle r_2 - r_1(n) - r_1(n-1:0) \rangle_{2^{n-1}} = \langle v_{2a} + v_{2b} + v_{2c} \rangle_{2^{n-1}} \\ v_3 &= \langle v_2 + r_3 - r_1 \rangle_{2^n} = \langle v_{3a} + v_{3b} + v_{3c} + \omega \rangle_{2^n} \end{aligned} \quad (7)$$

where $v_{2a} = r_2$, $v_{2b} = \bar{r}_{1(n-1:0)}$, $v_{2c} = \overbrace{1 \dots 1}^{n-1} \parallel \bar{r}_{1(n)}$, $v_{3a} = r_3$, $v_{3b} = \bar{r}_{1(n-2:0)} \parallel \bar{r}_{1(n)}$, $v_{3c} = r_2$, and ω is equal to 1, if $r_2 + \bar{r}_{1(n-1:0)} + \bar{r}_{1(n)} \geq 2^n$.

TABLE II: Two example for the ReLU operations according to the proposed schema.

$R = +12345 \equiv (60, 7, 25)$		
$r_2 = 00111$,	$\bar{r}_{1(4:0)} = 00011$,	$\bar{r}_{1(5)} = 1$
$r_3 = 11001$,	$\bar{r}_{1(3:0)} \parallel \bar{r}_{1(5)} = 00110$	
$r_2 + \bar{r}_{1(4:0)} + \bar{r}_{1(5)} = 01011 < 2^5 \rightarrow \omega = 0$		
$v_3 = \langle r_3 + \bar{r}_{1(3:0)} \parallel \bar{r}_{1(5)} + r_2 + \omega \rangle_{2^5} = 00110$		
MSB(v_3)=0 $\rightarrow R \geq 0 \rightarrow \text{OUT} = \{v_1, v_2, v_3\}$		
$R = -13579 \equiv (29, 30, 21)$		
$r_2 = 11110$,	$\bar{r}_{1(4:0)} = 00010$,	$\bar{r}_{1(5)} = 1$
$r_3 = 10101$,	$\bar{r}_{1(3:0)} \parallel \bar{r}_{1(5)} = 00101$	
$r_2 + \bar{r}_{1(4:0)} + \bar{r}_{1(5)} = 10001 \geq 2^5 \rightarrow \omega = 1$		
$v_3 = \langle r_3 + \bar{r}_{1(3:0)} \parallel \bar{r}_{1(5)} + r_2 + \omega \rangle_{2^5} = 11001$		
MSB(v_3)=1 $\rightarrow R < 0 \rightarrow \text{OUT} = \{0, 0, 0\}$		

2) *Activation*: The finding in [22] shows that the most significant bit (MSB) of v_3 identifies R 's sign, which is the main component of the implementation of the activation function (ReLU). In this step, $MSB(v_3)$ determines the proper output $\{0, 0, 0\}$ or $\{v_1, v_2, v_3\}$, negative or positive values, respectively, and send it back to the MRU to perform the rest of IOC operations. If it passes $\{0, 0, 0\}$, other operations are skipped, which leads to overall system performance improvements. Table II provides two examples for the operation of the ReLU architecture based on the proposed algorithm.

3) *Pooling*: Herein, we use MAX pooling, which is implemented by comparison operations. The magnitude comparison of two integers X and P in the residue domain² with moduli set $\{2^{n+1} - 1, 2^n - 1, 2^n\}$ can be carried out using their mixed-radix coefficients. Let (v_1^x, v_2^x, v_3^x) and (v_1^p, v_2^p, v_3^p) be the mixed-radix coefficients X and P , respectively. It first compares v_3^x with v_3^p , if they are equal, v_2 s will be compared, and if pair of v_3 s and v_2 s are equal, comparing v_1 s determines the comparison result, as shown in Fig. 3. This can be realized by employing three binary comparators along with Relational Inference. The $v_\alpha, v_\beta, v_\gamma$ are the maximum values between the applied inputs (v_i^x) and the stored values of the register file (v_i^p) , which are already compared. Then, the register is updated by the maximum values. After performing K -times³ pooling operations, the final output is taken by Scaler. Table III illustrates an example of one of the steps of the proposed pooling unit operation.

² P is the register file's output.

³ K equals the pooling window size, i.e., 2×2 .

TABLE III: An example for one step of the proposed RNS-based MAX-pooling operation.

$\mathbf{X} = 12345 \equiv (60, 7, 25)$	$\mathbf{P} = 11111 \equiv (23, 13, 7)$
$v_1^x = 60, v_2^x = 9, v_3^x = 6$	$v_1^p = 23, v_2^p = 21, v_3^p = 5$
$v_3^x > v_3^p \rightarrow \mathbf{X} > \mathbf{P} \rightarrow (v_\alpha, v_\beta, v_\gamma) = (60, 7, 25)$	

4) *Scaler*: In the final step of IOC, if the pooling's output is non-zero, the scaler normalizes the value to prevent possible overflow; otherwise, the normalization is skipped. The normalization of an integer R in RNS-based DNN also can be performed by using a scaling operation. The choice of scaling factor directly impacts the complexity of the scaling process and, consequently, normalization in the RNS-based DNN accelerator. Scaling by one modulus or the product of several moduli, scaling by an integer coprime with the moduli of RNS, and scaling by a power-of-two integer are three types of RNS scaling categories based on the scaling factor [21]. The hardware architecture of the Scaler unit is illustrated in Fig 3. Herein, to obtain a more efficient scaling, $2^{n+1} - 1$, as one of the targeted RNS moduli, is considered the scaling factor. According to this scaling factor and considering the definition of RNS scaling, the following holds:

$$S = \left\lfloor \frac{R}{2^{n+1} - 1} \right\rfloor = \left\lfloor \frac{v_1}{2^{n+1} - 1} + v_2 + (2^n - 1)v_3 \right\rfloor \quad (8)$$

$$= v_2 + (2^n - 1)v_3$$

The representation of the scaled integer S in the residue domain can be expressed as follows:

$$\begin{aligned} s_1 &= \langle S \rangle_{2^{n+1}-1} = \langle v_2 + (2^n - 1)v_3 \rangle_{2^{n+1}-1} \\ &= \langle v_3 || v_2 - v_3 \rangle_{2^{n+1}-1} \\ &= \langle 00 || v_{3(n-1:1)} + v_{3(0)} || v_2 + 1 || \bar{v}_3 \rangle_{2^{n+1}-1} \\ s_2 &= \langle v_2 + (2^n - 1)v_3 \rangle_{2^n-1} = v_2 \\ s_3 &= \langle v_2 + (2^n - 1)v_3 \rangle_{2^n} = \langle v_2 - \langle v_2 + r_3 - r_1 \rangle_{2^n} \rangle_{2^n} \\ &= \langle r_1 - r_3 \rangle_{2^n} \end{aligned} \quad (9)$$

An example of the normalization operation using the proposed scaling algorithm is summarized in Table IV.

TABLE IV: An example for the normalization is performed using the proposed approach in the RNSiM.

$\mathbf{R} =$	scaling factor =	expected scaled value =
23456 $\equiv (20, 20, 0)$	$2^6 - 1$	$(S) = \lfloor \frac{23456}{63} \rfloor = 372$
$v_1 = 010100$	$v_2 = 00000$	$v_3 = 01100$
$s_1 = \langle 00 v_{3(4:1)} + v_{3(0)} v_2 + 1 \bar{v}_3 \rangle_{2^6-1}$		
$= \langle 000110 + 000000 + 110011 \rangle_{2^6-1} = 111001 = 57_{decimal}$		
$s_2 = v_2 = 00000 = 0_{decimal}$		
$s_3 = \langle r_1(4:0) + \bar{r}_3 + 1 \rangle_{2^5} = \langle 10100 + 11111 + 1 \rangle_{2^5} = 10100 = 20_{decimal}$		

IV. PERFORMANCE EVALUATION

To validate and examine the functionality and performance of RNSiM, the same moduli set $\{2^6 - 1, 2^5 - 1, 2^5\}$ presented in [20] with the dynamic range $[0, 62496]$ is chosen, which is comparable with the dynamic range of 16-bit binary representation $[0, 65536]$.

A. Experiment Setup

• **Evaluation framework**: To assess the performance of the proposed accelerator, we create the evaluation testbed from scratch. At the circuit level, we develop RNSiM-CCU's sub-array with peripheral circuitry (sense amplifier, a modified row decoder, etc.) in Cadence Spectre with 45nm NCSU Product Development Kit (PDK) library [23] to achieve the performance parameters for the dual-row activation mechanism. The Synopsys Design Compiler with 45nm technology is leveraged to obtain the parameters of the IOC, i.e., energy and area, listed in Table V. Due to the MRU's complexity, it consumes the highest power and area, whereas the activation unit, using one 2-1 MUX, has the lowest values. We then configure the RNSiM's CCU memory sub-array with 1024 rows and 256 columns, 4×4 mats (with 1/1 as row/column activation) per bank organized in an H-tree routing manner, 16×16 banks (with 1/1 as row/column activation) in each memory group. It is worthy noted that, according to the unique number-theoretic properties of the adopted moduli-set, $\{2^{n+1} - 1, 2^n - 1, 2^n\}$ using by the RNSiM, the forward converter unit considering the size of ifmap only consists of two simplified modulo adders. This elegant simplicity provides a fast, energy-efficient, and cost-effective forward converter unit for RNSiM. In assessing and evaluating the proposed accelerator's performance, all the elements inside the RPU and other computational elements such as pre-processing unit (PPU), including a forward converter unit and a quantizer unit, are considered. The syntheses measures for the targeted moduli-set with $n=5$ reflect the relatively small overhead of the forward conversion (power= 26.53 μW , area= 127.03 μm^2) compared to other processing units in RNSiM architecture. Besides, without considering the inter-modulo operation unit, only CCU operates at least $K \times H \times W$ times⁴ more than a forward converter.

• **PIM platforms**: Different data-intensive applications with various workload sizes and memory access patterns are expected to benefit from PIM in both cache and main memory levels, so selecting the proper design for a particular application is a complex task. Besides, by choosing a PIM design, it is imperative to establish uniform evaluation conditions to make an impartial choice between available design options [24]. An identical physical memory configuration is considered for all the PIM implementations to perform a fair comparison with other DRAM PIM platforms. At the architecture level, we develop a comprehensive PIM compiler and evaluation framework working with the Cacti tool for RNSiM and three other processing-in-DRAM platforms, i.e., Ambit [6], DRISA-

⁴ K, H , and W are the number of filters, height, and width, respectively.

TABLE V: The power, delay, and area of the Inter-modulo Operation Core's components.

Units	Power (μW)	Area (μm^2)
Mixed-Radix Unit	57.72	205.86
Activation Unit	1.2	5.5
Pooling Unit	21.45	114.27
Normalization Unit	36.78	151.17

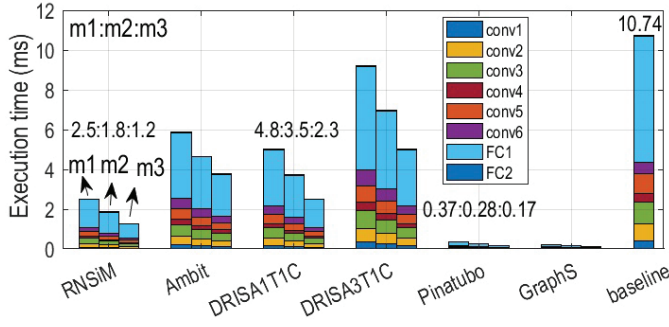


Fig. 4: The breakdown of execution time of RNSiM compared with RNS-Ambit, RNS-DRISA1T1C, RNS-DRISA3T1C, RNS-Pinatubo, RNS-GraphS, and non-RNS baseline 8-bit quantized network implemented by DRISA.

1T1C [8], DRISA-3T1C [8] with inputs from the circuit level simulations. To study the impact of the choice of non-volatile memory technologies in RNSiM's performance, we reimplemented the resistive memories, i.e., spin-transfer torque magnetic RAM (STT-MRAM)-based Pinatubo [25], and spin-orbit torque magnetic RAM (SOT-MRAM)-based GraphS [26] designs. Given an input network structure, the framework takes the instructions from a user interface, partitions, and maps the input data batches into the computational sub-arrays for three channels and then estimates the performance of PIM platforms. To the best of our knowledge, although RNSnet presented in [19] is the only RNS-based PIM platform that is implemented in non-volatile memory due to the different in-memory technologies and the lack of hardware details implementations, we excluded it from our assessment to make a fair comparison.

• **Model:** A CNN with six bit-wise convolutional layers, two max pooling, and two FC layers that cost about 80 FLOPs for a 40×40 image is chosen to run the SVHN dataset [27]. The images are re-sized to 40×40 and fed to the model.

B. Experiment Results

• **Execution time:** Figure 4 reports and compares the execution time of RNSiM with the under-test RNSed processing-in-DRAM platforms, i.e., Ambit [6], DRISA-1T1C, DRISA-3T1C [8], STT-Pinatubo [25], SOT-GraphS [26], and a non-RNS baseline implemented based on DRISA-3T1C. We observe that the RNSiM platform processes all the convolutional and fully-connected layers in less than 3 ms. It is noteworthy

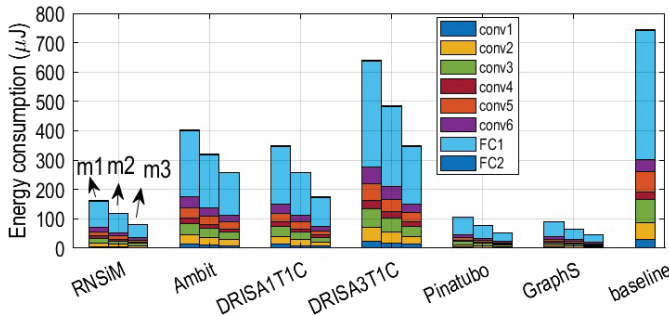


Fig. 5: The breakdown of energy consumption of RNSiM compared with RNS-Ambit, RNS-DRISA1T1C, RNS-DRISA3T1C, and non-RNS baseline 8-bit quantized network implemented by DRISA.

that as we are using three parallel channels corresponding to $\{m_1 = 63, m_2 = 31, m_3 = 32\}$ as depicted Fig. 4, the network execution time can be well estimated by the longest channel, i.e., m_1 . In this way, RNSiM reduces the execution time by a factor of $\sim 2.4\times$, $2\times$, and $3.6\times$ compared with RNSed Ambit, DRISA-1T1C, and DRISA-3T1C implementations. This advantage is achieved because (1) the proposed in-memory architecture is able to process the main components of adder trees based on CCU's instructions very efficiently in 1 or 2 cycles in parallel compared with multi-cycle operations in processing-in-DRAM counterparts and (2) the parallel mapping strategy matches the behavior of adder trees. Moreover, RNSiM achieves $\sim 4.3\times$ speed-up compared with the typical 8-bit quantized baseline implemented on DRISA-3T1C. Therefore, RNSiM could be a potential acceleration platform, especially in real-time applications, where processing speed is crucial. When it comes to non-volatile memory implementation, we can observe that the proposed RNS-based computational layers could be processed faster than DRAM. For instance, the magnetic Pinatubo implementation requires 0.37ms:0.28ms:0.17ms to process the whole layers in three channels. It is worth pointing out that the maximum latency was considered as the platform execution time. Therefore, STT and SOT designs could be a proper candidate for the digital write-back-intensive MAC-based applications such as CNNs due to the intrinsic STT/SOT-based magnetization switching.

• **Energy consumption:** We report the estimated energy consumption of various PIM platforms for running the same 8-layer network. The energy is reported by adding on the energy consumed in three parallel channels. Based on Fig. 5, RNSiM with the presented DRAM implementation in this work shows the least energy consumption ($361.21 \mu\text{J}$, total over three channels) to run the eight layers, as compared with the RNS-PIMs and the baseline platforms. The RNSiM reduces the energy consumption by $\sim 2.1\times$ compared with the DRISA1T1C implementation. Besides, it achieves $\sim 1.9\times$ lower energy vs. the baseline 8-bit quantized design. It is worth pointing out that non-RNS PIM consumes less energy compared with the RNS-DRISA3T1C. Fig. 5 also depicts the relative comparison between the volatile and the non-volatile in-memory implementation of RNSiM. It can be observed that the fully-RNS-based computational layers could be processed more energy-efficiently if implemented with a non-volatile memory candidate such as GraphS [26].

• **Standing against von-Neumann architecture:** To further explore the performance of RNSiM and compare it with von-Neumann computing architecture, we design an experiment to run the same 8-layer CNN structure over FPGA and GPU platforms. We take a Xilinx PYNQ-Z1 FPGA board-XC7Z020 (53,200 LUTs, 106,400 FFs, 280 18Kb BRAMs, 220 DSP48Es) and an NVIDIA GTX 1080 Ti GPU to implement an 8-bit quantized network running the same SVHN dataset. The results are tabulated in Table VI. We observe the considerable energy-efficiency of RNS-PIM platforms vs. FPGA and GPU platforms. RNSiM and other RNS-PIMs reduce the energy consumption by orders of $28\text{-}77\times$ and

TABLE VI: Performance results of PIM vs. non-PIM platforms for processing a single image.

Platforms	Network/Dataset	Energy ($\mu\text{J}/\text{img}$)	Latency (ms/img)
RNSiM	RNS($m=5$)/SVHN	361.21	2.51
Ambit	RNS($m=5$)/SVHN	978.1	5.85
DRISA-1T1C	RNS($m=5$)/SVHN	777.6	5.02
Xilinx PYNQ Z1 FPGA	Quantized($n=8$)/SVHN	27918	8.33
NVIDIA GTX 1080 Ti GPU	Quantized*($n=8$)/SVHN	324000	90

*We scaled the floating-point GPU results by $4\times$ to report the 8-bit quantized network performance since no framework supports fixed-point CNN on GPU.

$331\text{--}897\times$ compared with the FPGA and the GPU platforms, respectively. This is mainly due to the energy-efficient in-memory operations of PIM platforms that totally eliminate the need for external data communication. Table VI also reports the required time to process the whole CNN layers. Based on the results, RNSiM achieves $3.3\times$ and $35.8\times$ speed-up vs. the FPGA and GPU.

The time fraction at which on-/off-chip data transfer limits the RNS performance and the processing unit needs to wait for data is referred to as memory bottleneck ratio (MBR). We reported the MBR for different platforms listed in Table VI. This evaluation is accomplished through experimentally extracted results for each platform regarding the number of memory accesses. As we observe in Table VII, PIM solutions spend less than 21% time for data transfer and memory access. But, the FPGA and GPU solutions spend over 74% time waiting for the loading data from memory. In this way, a resource utilization ratio (RUR) can be defined for various platforms. Based on this, RNSiM and Ambit implementations obtain the highest ratio by utilizing up to $\sim 80\%$ of their computation resources, where GPUs show 5% performing the identical task.

TABLE VII: Estimated memory bottleneck ratio and resource utilization.

Platforms	MBR (%)	RUR (%)
RNSiM	22	78
Ambit	21	79
DRISA-1T1C	24	76
Xilinx PYNQ Z1 FPGA	74	26
NVIDIA GTX 1080 Ti GPU	95	5

C. Accuracy Exploration

In order to perform a fair comparison between our design (5-bit RNS-CNN) and three other NN models, 32-bit full-precision (FP), 8-bit fixed and 1-bit binarized neural network (BNN) models, the same architecture, including six convolutional layers, and two FC layers is leveraged. To evaluate the accuracy, we first train the CNN with the 32-bit floating-point number, and then the pre-trained weights are used for all the examined CNN models. We conduct experiments of RNS-CNN on several datasets, including MNIST [28], SVHN [27], and CIFAR-10 [29]. PyTorch implementation of RNS-CNN inspired by DoReFa-Net [1], with the difference that our design includes both BNS and RNS -based components. Since there is no accuracy loss in the CONV layers, we use the actual convolutional operations (BNS) for this step. The produced partial sums, or output feature maps, are converted to the RNS

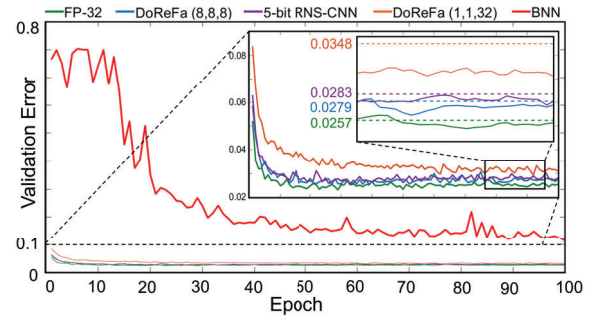


Fig. 6: Validation error curves of five different configurations on SVHN.

(BNS-to-RNS), and all the required operations within the IOC are performed in the RNS domain before the next convolution begins. Finally, the backward conversion (RNS-to-BNS) is utilized to prepare input of the next convolutional operations. Figure 6 shows the validation error (VE) versus the number of epochs of four different networks on the SVHN dataset. The training convergence behaviors for all the examined NNs except fully-binarized NN are achieved almost after ten epochs. Although 32-bit FP shows the lowest VE value (with an average of 0.025), RNS-CNN shows better and acceptable accuracy than BNN. Table VIII compares and summarizes the classification accuracy for three different datasets. In addition to the 32-bit full-precision as the baseline, RNS-CNN, and two other networks, including 8-bit fixed and binarized neural networks, are evaluated. Based on the obtained results, the RNS-CNN shows a small accuracy degradation while provides significant energy-product-delay reduction.

TABLE VIII: Inference accuracy (%) on MNIST, SVHN and CIFAR-10 datasets.

Configuration	MNIST	SVHN	CIFAR-10
32-bit FP	99.46	97.52	91.25
8-bit fixed	99.38	96.9	93.1
5-bit RNS-CNN (ours)	99.05	96.2	91.7
BNN	98.60	91.1	89.8

V. CONCLUSIONS

In this paper, we presented a novel hardware-optimized accelerator using the Residue Number System called RNSiM to improve the energy efficiency and delay of CNNs. RNSiM is mainly developed based on the enhanced integrated inter-modulo operation core and PIM architecture to reduce the overhead imposed by non-modular operations and accelerate RNS-based multiplication-and-accumulation. The computational performance of RNSiM exploited its intrinsic in-memory parallelism and parallelised RNS channels to increase the system's throughput. The obtained simulation results showed that our RNSiM and other RNS-PIMs based on our method reduce the energy consumption by orders of $28\text{--}77\times$ and $331\text{--}897\times$ compared with the FPGA and the GPU platforms, respectively. The RNS-based CNN's accuracy using RNSiM has been evaluated on different datasets, including MNIST, SVHN, and CIFAR-10. Due to the small accuracy degradation compared to the 32-bit full-precision and a significant reduction in the hardware aspect, the RNSiM is a promising inference scheme for power-constrained edge devices.

REFERENCES

- [1] S. Zhou, Y. Wu, Z. Ni, X. Zhou, H. Wen, and Y. Zou, "Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients," *arXiv preprint arXiv:1606.06160*, 2016.
- [2] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "Xnor-net: Imagenet classification using binary convolutional neural networks," in *European conference on computer vision*. Springer, 2016, pp. 525–542.
- [3] C.-H. Chang, A. S. Molahosseini, A. A. E. Zarandi, and T. F. Tay, "Residue number systems: A new paradigm to datapath optimization for low-power and high-performance digital signal processing applications," *IEEE circuits and systems magazine*, vol. 15, pp. 26–44, 2015.
- [4] J. L. Gustafson and I. T. Yonemoto, "Beating floating point at its own game: Posit arithmetic," *Supercomputing Frontiers and Innovations*, vol. 4, no. 2, pp. 71–86, 2017.
- [5] P. Chi, S. Li, C. Xu, T. Zhang, J. Zhao, Y. Liu, Y. Wang, and Y. Xie, "Prime: A novel processing-in-memory architecture for neural network computation in reram-based main memory," *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 27–39, 2016.
- [6] V. Seshadri, D. Lee, T. Mullins, H. Hassan, A. Boroumand, J. Kim, M. A. Kozuch, O. Mutlu, P. B. Gibbons, and T. C. Mowry, "Ambit: In-memory accelerator for bulk bitwise operations using commodity dram technology," in *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2017, pp. 273–287.
- [7] A. Roohi, S. Sheikhaal, S. Angizi, D. Fan, and R. F. DeMara, "Apgan: Approximate gan for robust low energy learning from imprecise components," *IEEE Transactions on Computers*, vol. 69, no. 3, pp. 349–360, 2019.
- [8] S. Li, D. Niu, K. T. Malladi, H. Zheng, B. Brennan, and Y. Xie, "Drisa: A dram-based reconfigurable in-situ accelerator," in *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2017, pp. 288–301.
- [9] S. Angizi and D. Fan, "Redram: A reconfigurable processing-in-dram platform for accelerating bulk bit-wise operations," in *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2019, pp. 1–8.
- [10] M. Taheri, A. S. Molahosseini, and K. Navi, "Efficient incorporation of the rns datapath in reverse converter," *IEEE Transactions on Circuits and Systems II: Express Briefs*, 2020.
- [11] M. Abdelhamid and S. Koppula, "Applying the residue number system to network inference," *arXiv preprint arXiv:1712.04614*, 2017.
- [12] Z.-G. Liu and M. Mattina, "Efficient residue number system based winograd convolution," *arXiv preprint arXiv:2007.12216*, 2020.
- [13] H. Nakahara and T. Sasao, "A high-speed low-power deep neural network on an fpga based on the nested rns: Applied to an object detector," in *2018 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2018, pp. 1–5.
- [14] H. Nakahara and t. Sasao, "A deep convolutional neural network based on nested residue number system," in *2015 25th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 2015, pp. 1–6.
- [15] G. Alia and E. Martinelli, "Neurom: a rom based rns digital neuron," *Neural networks*, vol. 18, no. 2, pp. 179–189, 2005.
- [16] V. Arrigoni, B. Rossi, P. Fragneto, and G. Desoli, "Approximate operations in convolutional neural networks with rns data representation," in *ESANN*, 2017.
- [17] N. I. Chervyakov, P. A. Lyakhov, M. V. Valueva, G. V. Valuev, D. I. Kaplun, G. A. Efimenko, and D. V. Gnezdilov, "Area-efficient fpga implementation of minimalistic convolutional neural network using residue number system," in *2018 23rd Conference of Open Innovations Association (FRUCT)*. IEEE, 2018, pp. 112–118.
- [18] N. Chervyakov, P. Lyakhov, M. Deryabin, N. Nagornov, M. Valueva, and G. Valuev, "Residue number system-based solution for reducing the hardware cost of a convolutional neural network," *Neurocomputing*, 2020.
- [19] S. Salamat, M. Imani, S. Gupta, and T. Rosing, "Rnsnet: In-memory neural network acceleration using residue number system," in *2018 IEEE International Conference on Rebooting Computing (ICRC)*. IEEE, 2018, pp. 1–12.
- [20] N. Samimi, M. Kamal, A. Afzali-Kusha, and M. Pedram, "Res-dnn: A residue number system-based dnn accelerator unit," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 67, no. 2, pp. 658–671, 2019.
- [21] M. Taheri, K. Navi, and A. Sabbagh Molahosseini, "Efficient programmable power-of-two scaler for the three-moduli set $\{2n+ p, 2n-1, 2n+ 1- 1\}$," *ETRI Journal*, vol. 42, no. 4, pp. 596–607, 2020.
- [22] C. V. Niras and Y. Kong, "Fast sign-detection algorithm for residue number system moduli set $\{2 n- 1, 2 n, 2 n+ 1- 1\}$," *IET Computers & Digital Techniques*, vol. 10, no. 2, pp. 54–58, 2016.
- [23] (2011) Ncsu eda freepdk45. [Online]. Available: <http://www.eda.ncsu.edu/wiki/FreePDK45:Contents>
- [24] S. Angizi, Z. He, D. Reis, X. S. Hu, W. Tsai, S. J. Lin, and D. Fan, "Accelerating deep neural networks in processing-in-memory platforms: Analog or digital approach?" in *2019 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. IEEE, 2019, pp. 197–202.
- [25] S. Li, C. Xu, Q. Zou, J. Zhao, Y. Lu, and Y. Xie, "Pinatubo: A processing-in-memory architecture for bulk bitwise operations in emerging non-volatile memories," in *Proceedings of the 53rd Annual Design Automation Conference*, 2016, pp. 1–6.
- [26] S. Angizi, J. Sun, W. Zhang, and D. Fan, "Graphs: A graph processing accelerator leveraging sot-mram," in *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2019, pp. 378–383.
- [27] Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, and A. Y. Ng, "Reading digits in natural images with unsupervised feature learning," 2011.
- [28] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [29] A. Krizhevsky and G. Hinton, "Convolutional deep belief networks on cifar-10," *Unpublished manuscript*, vol. 40, no. 7, pp. 1–9, 2010.